MuscleMap — Build Brief for Claude Code

> "MuscleMap" is a working title. Keep the name in one constant (`APP_NAME`) so it can be renamed in one place.

---

0. How to work on this project

You are building a production mobile app from this brief, agentically, across many sessions.

1. **First session: plan mode.** Read this entire document. Produce `docs/PLAN.md` (phase breakdown, proposed file structure, open questions) and a root `CLAUDE.md` that distills §3 (stack), §4.3 (engine rules) and §7 (conventions). Ask your open questions, then start Phase 0.
2. **Work phase by phase (§8).** Do not start a phase until the previous phase's definition of done passes. At the end of every phase, rewrite `docs/STATUS.md` (done / stubbed / known bugs / next steps) so a fresh session can pick up cold.
3. **Log decisions.** Every non-obvious technical choice goes in `docs/DECISIONS.md` as a one-paragraph ADR.
4. **Commit small and often** with conventional-commit messages. Never leave the repo failing typecheck, lint, or tests.
5. **Stop and ask a human before:** adding any paid service or API key; adding a native module that forces a new dev build; changing any science constant in §4.3; anything that touches user photos; switching the 3D rendering approach away from the §6.2 default without a failed, documented spike.
6. **The science in §4 is the spec.** Do not "improve" it from general knowledge. If you believe something is wrong, write the objection in `docs/DECISIONS.md` and ask before changing it.
7. **The humans:** Avi (product owner; EMT / pre-med, trains science-based hypertrophy, develops on Windows and tests on an Android Pixel) and Nuri (co-founder). **Assume no Mac is available.** iOS builds go through EAS Build; iOS testing goes through TestFlight.

---

1. What this app is

A workout logger and history tracker with feature parity to Strong, plus one thing nobody has done well: a **Muscle Map** — a 3D anatomical model the user drags to rotate, where every muscle region is colored by how well the user's *actual logged training* covers it, computed from evidence-based weekly-volume rules — and a **weekly report** that says exactly what to add, move, or cut.

Core loop: log a workout → every set credits the muscles it trains → the map recolors → tap a muscle for detail, history, and trusted technique videos → the weekly report tells you what to change.

Audience: beginners first (the map teaches them what a balanced program is), but the engine must hold up for intermediate and advanced lifters (higher volume targets, program import, editable targets, no hand-holding).

Platforms: iOS and Android from one codebase.

---

2. Non-negotiables

- **Offline-first.** Full logging, history, and map with no account and no network. Sync is additive, never required.
- **Fast logging.** A set is loggable in ≤2 taps from the workout screen. Strong is the bar; any flow slower than Strong is wrong.
- **Deterministic engine.** Muscle status and report recommendations come from pure, tested functions over logged data. An LLM may write prose *around* the numbers; it never produces the numbers.
- **Science-based framing in all copy.** Say "weekly hard sets", "stimulus", "coverage". Never "losing gains", "shock the muscle", "muscle confusion", "toning". Colors reflect **programming coverage**, not muscle size — a red muscle has not been trained recently; it has not "shrunk". Real detraining takes weeks, and the copy must never imply otherwise.
- **Never lose a set.** Auto-save every set; recover an in-progress workout on relaunch.
- **Privacy.** Training data belongs to the user. Photos (if the feature ships, §9) stay on-device unless the user explicitly opts into cloud backup. Data export (CSV + JSON) is mandatory, not a nice-to-have.
- **The 3D map ships in the first build, with both a female and a male body.** No 2D placeholder body map. The 3D rendering spike happens in Phase 0 so

that risk is retired before anything else is built, and both anatomical models are produced in Phase 1.

- **No wall of red on day one.** See intake (§5) and provisional stimulus.
- **Color is never the only signal** (color-blind users): every status also has an icon/pattern and text.

---

3. Stack (decided — don't relitigate without a written ADR)

Concern	Choice
App	Expo (React Native), TypeScript `strict`, `expo-router`, EAS Build + EAS Update
State	Zustand for UI/session state; the database is the source of truth
Database	`expo-sqlite` + Drizzle ORM, versioned migrations, repository layer in `src/db`
3D body map	`@react-three/fiber` **v9, pinned** (`@react-three/fiber/native` entry) on `expo-gl`, `three` at the version fiber v9 supports, GLB loaded with `three-stdlib`'s `GLTFLoader` from an `expo-asset` URI. Do **not** use fiber v10 / `@react-three/native` — it was flagged unstable at the time of writing; any upgrade needs an ADR. Gestures via `react-native-gesture-handler` + `react-native-reanimated` (no `OrbitControls` — it needs DOM events). **Fallback only if the Phase 0 spike fails on a real device:** `react-native-webview` hosting a bundled three.js page with `OrbitControls`, `postMessage` bridge for colors in / taps out. Either way, one component: `MuscleMapView` with identical props.
3D asset pipeline	Blender headless (`blender --background --python`) or the `bpy` pip wheel; sources per §6.3; outputs `assets/models/body-female.glb` and `assets/models/body-male.glb`, both validated in CI

Concern	Choice
Charts	`victory-native` (fall back to `react-native-gifted-charts` if Victory fights Expo; decide once, log it)
Backend (Phase 4+)	Supabase — Auth, Postgres for sync, Edge Functions for anything needing a secret (YouTube key, Anthropic key). **The client never holds secrets.**
Video	YouTube Data API v3 via an edge function with caching; `react-native-youtube-iframe` for playback
LLM	Anthropic API via edge function, used only for (a) parsing free-text programs into structured JSON and (b) narrating the report. Schema-validated output, one retry, then fall back to manual/plain.
Testing	Jest (`jest-expo`) for engine + reducers; `@testing-library/react-native` for components; Maestro flows for the top 5 journeys once UI stabilizes
CI	GitHub Actions: typecheck, ESLint + Prettier, tests on every PR
Notifications / haptics	`expo-notifications` (rest timer), `expo-haptics`
Exercise seed data	`yuhonas/free-exercise-db` on GitHub (Unlicense; ~870 exercises with primary/secondary muscles, instructions, images). Import once into the taxonomy in §4.1, then assign contribution weights per §4.2. Verify the LICENSE file at import time and record it in `docs/DECISIONS.md` .

---

4. Domain model and the muscle engine

4.1 Muscle taxonomy

Two levels. **Groups** are the units that volume landmarks apply to — they get scored. **Regions** are sub-areas inside a group that specific exercises emphasize — they get an "emphasis" check and their own color on the map. IDs are stable `snake_case` strings stored in the DB; display names live in a strings file.

Group id	Regions	Notes
`chest`	`chest_upper` (clavicular), `chest_mid` (sternal), `chest_lower` (costal)	Incline work shifts share to upper
`delt_front`	—	Heavily trained by all pressing; low direct-volume target
`delt_side`	—	Needs direct work (lateral raises etc.)
`delt_rear`	—	Rows/pulldowns credit it partially; needs direct work
`lats`	—	
`upper_back`	`traps_mid_lower`, `rhomboids`	Rows, face pulls, shrug variants at depression/retraction
`traps_upper`	—	Shrugs, carries; low target
`erectors`	—	Hinges, squats, extensions; low target
`biceps`	`biceps`, `brachialis`	Brachialis emphasized by hammer/neutral-grip curls
`triceps`	`triceps_long`, `triceps_lateral_medial`	Long head emphasized by overhead extensions
`forearms`	`forearm_flexors`, `forearm_extensors_brachioradialis`	Low target; heavily indirect
`abs`	—	
`obliques`	—	

Group id	Regions	Notes
`glutes`	`glute_max`, `glute_med`	Med emphasized by abduction / single-leg work
`quads`	`quads_rf` (rectus femoris), `quads_vasti`	RF is a hip flexor — squats barely train it; leg extensions / sissy squats do
`hamstrings`	`hamstrings_hip_ext` (semis + BF long head), `hamstrings_bf_short`	BF short head is knee-flexion only — needs a curl variant
`adductors`	—	
`calves`	`gastrocnemius`, `soleus`	Gastroc = straight-knee; soleus = bent-knee
`tibialis`	—	Off by default; opt-in
`neck`	`neck_flexors`, `neck_extensors`	Off by default; opt-in
`hip_flexors`	—	Off by default; opt-in

4.2 Exercise → muscle mapping

Each exercise maps to one or more groups with a **contribution weight**, plus an optional **region distribution** inside each group.

- Primary movers: weight `1.0`. Secondary/indirect: `0.5` by default (tunable `0.25` – `1.0`).
- Rule: **indirect work counts at roughly half credit.** A row credits `biceps` at 0.5 — biceps aren't red after a back day, but rows alone won't turn them green.
- Region distribution is a share of the group credit and must sum to 1.0.

Examples (seed these; the humans will review the full table):

Exercise	Group credits	Region distribution
Barbell bench press	chest 1.0, delt_front 0.5, triceps 0.5	chest → upper 0.25 / mid 0.50 / lower 0.25; triceps → long 0.3 / lat_med 0.7
Incline DB press	chest 1.0, delt_front 0.5, triceps 0.5	chest → upper 0.60 / mid 0.35 / lower 0.05
Barbell row	lats 0.75, upper_back 1.0, delt_rear 0.5, biceps 0.5, erectors 0.25	—
Lat pulldown	lats 1.0, upper_back 0.5, biceps 0.5, delt_rear 0.25	—
Back squat	quads 1.0, glutes 0.75, adductors 0.5, erectors 0.5	quads → rf 0.10 / vasti 0.90; glutes → max 0.9 / med 0.1
Romanian deadlift	hamstrings 1.0, glutes 0.75, erectors 0.5	hamstrings → hip_ext 1.0 / bf_short 0.0
Seated leg curl	hamstrings 1.0	hamstrings → hip_ext 0.6 / bf_short 0.4
Overhead triceps extension	triceps 1.0	triceps → long 0.7 / lat_med 0.3
Lateral raise	delt_side 1.0	—
Standing calf raise	calves 1.0	calves → gastroc 0.7 / soleus 0.3
Seated calf raise	calves 1.0	calves → gastroc 0.2 / soleus 0.8

Mappings are **data** (`src/data/exercise-muscles.json`), loaded into the DB by migration, with a validation test (weights in range, distributions sum to 1.0, every exercise has ≥1 primary). Custom exercises: the user picks primary and secondary groups (and optionally a region emphasis) in a picker; weights are assigned by role.

4.3 The engine — `src/engine/` (pure TypeScript, no React, no `Date.now()`, 100% tested)

Inputs: all sets in the last 14 days (`exerciseId, reps, weight, rir?, setType, performedAt, workoutId, isProvisional`), the user's level, priority groups, target overrides, and `now` (injected).

Per-set factors:

```
// Hardness: proximity to failure. Warm-ups never count.
hardness(set):  setType === 'warmup' ? 0
              : set.rir == null     ? 1.0   // unlogged RIR =
assume a working set
              : set.rir <= 3        ? 1.0
              : set.rir === 4       ? 0.5
              :                       0     // ≥5 RIR is not a
stimulating set
// Drop sets / myo-rep clusters count as ONE set, not one per
portion.

// Rep-range factor: ~5–30 reps to near failure grow similarly;
very low reps are less efficient per set.
repFactor(set): reps < 3 ? 0.5 : reps < 5 ? 0.75 : reps <= 30 ?
1.0 : 0.5

// Recency: rolling week with a 3-day taper so the map fades
instead of flipping at midnight on day 8.
recency(daysAgo): daysAgo <= 7 ? 1.0 : daysAgo <= 10 ? 1 -
(daysAgo - 7) / 3 : 0

// Per-session diminishing returns ("junk volume"). Applied per
group, per workout,
// over that group's credited sets ordered by time: sets 1–8
count 1.0, 9–12 count 0.5, 13+ count 0.25.
sessionFactor(indexWithinWorkout)
```

Group score:

```
effectiveSets(group) = Σ over sets: weight(exercise, group) ×
hardness × repFactor × sessionFactor × recency
frequency(group)     = number of distinct workout days in the
last 7 days with ≥ 0.5 credited set for that group
```

Targets (weekly effective sets per group, by level) and **group multipliers** (small or heavily-indirect muscles need less direct volume):

```
export const TARGETS = {
  beginner:     { min: 4, target: 8,  high: 12 },   // < 1 year
consistent
  intermediate: { min: 6, target: 12, high: 18 },   // 1–3 years
  advanced:     { min: 8, target: 15, high: 22 },   // 3+ years
};
export const GROUP_MULTIPLIER = {
  delt_front: 0.5, forearms: 0.5, traps_upper: 0.5, erectors:
0.5,
  abs: 0.75, obliques: 0.75, adductors: 0.75, tibialis: 0.5,
neck: 0.5, hip_flexors: 0.5,
  // everything else: 1.0
};
// Priority groups (user-selected in intake): multiplier × 1.25.
// Advanced users may override min/target/high per group in
Settings; show the defaults and the rationale.
```

Status: with `T = TARGETS[level] × GROUP_MULTIPLIER[group] × priority`, and `r = effectiveSets / T.target`:

Condition	Status	Color
`effectiveSets === 0`	`untrained`	red
`0 < effectiveSets < T.min`	`low`	red → orange gradient
`T.min ≤ effectiveSets < T.target`	`partial`	yellow → yellow-green gradient
`effectiveSets ≥ T.target`	`covered`	green
`effectiveSets > T.high`	`covered` + flag `high_volume`	green (flag shown in detail + report)
`effectiveSets > 1.3 × T.high`	flag `possibly_excessive`	green + warning badge

Color = continuous interpolation on `r` (red at 0, yellow at 0.5, green at ≥1.0) in OKLCH or HSL so midpoints read as yellow, not mud. Clamp at 1.0.

Region emphasis (the "every muscle singled out" layer): each region's effective sets = its share of the group's credited sets via the distributions. A region is `under_emphasized` when its share is below `region.minShare` while the group has ≥ `T.min` sets. Region color = its group's color, dimmed one step when under-emphasized, with a badge and a concrete fix in the detail view ("chest_upper: 12% of chest volume — add an incline press").

```
export const REGION_MIN_SHARE = {
  chest_upper: 0.25, chest_lower: 0.10, triceps_long: 0.30,
hamstrings_bf_short: 0.20,
  soleus: 0.25, quads_rf: 0.10, glute_med: 0.10, brachialis:
0.15,
  // regions not listed: no emphasis check
};
```

The engine is sex-agnostic. The evidence does not support different volume landmarks or set-counting rules by sex, so the female/male choice only selects which body is rendered. Do not add sex-specific targets; if someone asks, that is an ADR-and-ask.

Other engine outputs (used by the report, not by color):

- `recentlyTrained(group)` : ≥ 4 credited sets in the last 48 h → "trained recently" indicator and excluded from "what to train today".
- `distributionAdvice(group)` : `effectiveSets ≥ 10 && frequency === 1` → "split this across two days".
- `overloadStatus(exercise)` : for exercises with ≥ 4 sessions in the last 6 weeks, best e1RM (Epley: `w × (1 + reps/30)`) and best volume-load per session; flag `stalled` if neither improved across the last 4 sessions.
- `balanceFlags` : push:pull ratio (`chest + delt_front + triceps` vs `lats + upper_back + delt_rear + biceps`) outside 0.7–1.3; `quads:hamstrings` outside 0.6–1.6; `delt_front` more than 2× `delt_side` or `delt_rear` .
- `adherence` : planned sets (templates scheduled this week) vs completed.

Keep all constants in `src/engine/constants.ts` with a comment citing the source of each. Starting points (verify the citations yourself before quoting them in-app): Schoenfeld, Ogborn & Krieger 2017 (weekly volume dose-response); Schoenfeld, Ogborn & Krieger 2016 (frequency); Baz-Valle et al. 2022 (high-volume ceiling); Pelland et al. 2024 preprint (volume dose-response meta-regression); Refalo et al. 2023 (proximity to failure); Schoenfeld et al. 2017 (low

vs high load); Maeo et al. 2022 (triceps long head, overhead); Chaves et al. 2020 (incline angle and upper chest); practitioner syntheses from Jeff Nippard's volume-landmark videos and RP's MEV/MAV/MRV framework.

4.4 Data model (Drizzle)

- `profile` (single local row): `level`, `units`, `goal`, `equipmentProfile`, `priorityGroups[]`, `targetOverrides` JSON, `enabledOptionalGroups[]`, `createdAt`.
- `exercises`: `id`, `name`, `aliases[]`, `equipment`, `mechanic` (compound/isolation), `pattern` (push/pull/hinge/squat/carry/…), `instructions`, `mediaRefs`, `isCustom`, `source`.
- `exercise_muscles`: `exerciseId`, `groupId`, `weight`, `regionDistribution` JSON.
- `workouts`: `id`, `startedAt`, `endedAt`, `name`, `templateId?`, `notes`, `isProvisional`.
- `workout_exercises`: `id`, `workoutId`, `exerciseId`, `order`, `supersetGroup?`, `notes`.
- `sets`: `id`, `workoutExerciseId`, `order`, `weight`, `reps`, `rir?`, `rpe?`, `setType` (`warmup|working|drop|failure|myo`), `isCompleted`, `performedAt`.
- `templates`, `template_exercises`, `template_sets` (target weight/reps/RIR), `template_schedule` (weekday assignments, optional).
- `personal_records` (cached, recomputed on write).
- `muscle_status_cache`: `groupId` / `regionId`, `computedAt`, `effectiveSets`, `ratio`, `flags[]` — recomputed after every set save (must be < 50 ms on one year of history).
- `bodyweight_log`.
- `video_cache`: `key` (exerciseId or groupId), `videoId`, `channelId`, `title`, `fetchedAt`.
- `photos` (Phase 7 only): `id`, `localUri`, `takenAt`, `poseTag`.

---

5. Intake / onboarding

Goal: classify level, seed the map so it is useful on day one, take ≤ 3 minutes. Every step skippable except level.

1. **Basics:** body model (female / male — which figure the map shows; changeable in Settings, no effect on the engine); units; optional bodyweight/height.
2. **Experience:** "How long have you trained with weights consistently (≥ 2×/week, no break longer than a month)?" → never / < 6 mo / 6–12 mo / 1–3 y / 3 y+. Two calibration questions (do you track RIR/RPE? do you follow a written program?). Level = beginner (< 1 y), intermediate (1–3 y), advanced (3 y+). Editable later in Settings.
3. **Goal & equipment:** hypertrophy / strength / general; commercial gym / home barbell / dumbbells only / bands + bodyweight. Affects exercise recommendations and starter templates only.
4. **Priority muscles:** tap groups on the 3D model (same `MuscleMapView` in a selection mode) that the user wants to bring up → `priorityGroups` (multiplier × 1.25; report emphasizes them). This replaces photo analysis in v1 (see §9).
5. **Program import (intermediate/advanced only):** "What does your current program look like?" Two paths:
   - Structured builder: days → exercises → sets × reps.
   - Free text ("Push: bench 4×8, incline DB 3×10, lateral raises 4×15…") → LLM parses to a schema-validated JSON program → user confirms/edits.
     Save it as templates **and** generate **provisional stimulus**: simulate the last 7 days as if the program had been run at RIR 2, stored as `isProvisional = true`. Provisional data colors the map (rendered hatched/dimmed with a banner "estimated from your program — log workouts to replace"), decays like real data, and is excluded from PRs and progress charts. Real logged sets replace it.
6. **Beginner path:** no program step. Offer three starter templates — Full Body 3×/week, Upper/Lower 4×/week, Push/Pull/Legs — each built so that following it makes every default group reach ≥ `target` on beginner numbers. Verify this with an engine test.
7. Notifications opt-in → land on the Log tab with templates ready.

---

6. Screens

Tabs: **Log · History · Map · Progress · Settings**. The weekly Report lives inside Map (prominent button) and on the Home/Log header when a new one is

available.

6.1 Log (Strong parity)

Start from a template or empty. Add exercise: search with aliases, filter by group/equipment/pattern, recently used, "hits this muscle" filter. Set rows: weight / reps / RIR quick entry, previous performance ghosted in, tap to complete, warm-up toggle, set-type menu (drop / failure / myo). Auto rest timer with notification + haptic, per-exercise defaults. Supersets. Notes per exercise and per workout. Plate calculator. Unit toggle. PR toast on e1RM / rep / volume PRs. Finish screen shows a summary **plus the Muscle Map recoloring** (compact, auto-rotating front→back, non-interactive) with a list of which groups moved toward green. Edit or delete past workouts.

6.2 Map (the differentiator) — 3D from day one

A full-screen 3D écorché (skinless, muscles visible) that the user drags to rotate, colored by status. Legend; groups ↔ regions toggle; "What to train today" chip (groups furthest below target, excluding `recentlyTrained`); Report button.

Interaction spec

- One-finger drag: rotate. Yaw unbounded around the model's vertical axis; pitch clamped to ±25°. Inertia on release with friction decay (~1.5 s to rest).
- Pinch: zoom. Camera distance clamped so the model is never clipped and never smaller than ~60% of the viewport height.
- Two-finger drag: pan vertically (inspect calves or neck while zoomed). Double-tap: reset to the front view.
- Snap buttons: Front · Back · Left · Right (animated 300 ms).
- Tap (movement < 8 px, duration < 250 ms, so it never fires mid-drag): raycast → region id → highlight (emissive pulse, all other regions dimmed ~30%) → bottom sheet. Selection persists while the sheet is open; tap empty space or close the sheet to clear.
- Status color changes animate (lerp ~400 ms). The finish-workout screen reuses this so the user watches the model recolor.
- Provisional (estimated) data: desaturated + ~60% opacity, "estimated" label in the sheet.
- `frameloop="demand"`: render only on gesture, animation, or state change — this is a battery app, not a game.

- Group mode colors every region of a group identically; region mode shows emphasis dimming per §4.3.

Look. MatCap shading with a per-region color multiply — cheap, identical across GPUs, no lighting setup to tune. Subtle rim so adjacent regions read as separate. Untracked structures (`body_base` : deep muscles, skull, hands, feet, optional low-poly skeleton, and on the female body the mammary tissue form) in neutral gray. Female and male bodies ship together; only the selected one is loaded, and switching swaps the asset without touching the scene code.

Budget. ≤ 150k triangles total, ≤ 8 MB GLB, one mesh per region (≈ 35 draw calls). 60 fps during rotation on a mid-range Android device; cold load < 1.5 s (preload the asset while the app boots). If GL initialization fails on a device, show the grouped status list from Settings → "Muscle status (list)" with the same colors; that list exists anyway for accessibility.

Tap → bottom sheet

- status bar (effective sets this week vs target, ratio), sessions this week, last trained, flags;
- 8-week sparkline of weekly effective sets;
- exercises from the user's history that credited it, with dates and best sets;
- recommended exercises for the user's equipment (ranked by weight, then novelty vs. what they already do);
- 3–5 technique videos (§6.6), or nothing if none pass the allowlist.

Asset contract (`assets/models/body-female.glb` , `assets/models/body-male.glb` — identical contract, identical region set): glTF 2.0 binary; one mesh per region named *exactly* by region id (`chest_upper` , `quads_rf` , …), plus `body_base` ; left and right sides joined into the one region mesh; neutral gray materials (colors set at runtime); Y-up; stature normalized to 1.0 unit with feet at y = 0; origin at the model's center; no textures; Draco optional and only if native decoding proves reliable in the Phase 0 spike. `model.manifest.json` beside it lists regions, triangle counts, source revision, and license.

6.3 Model pipeline — `tools/model-pipeline/` (you build and run this; nobody hand-models anything)

The app ships **two bodies from the first build: female and male.** Both are produced by this pipeline, both satisfy the same asset contract, and both must look like they came from the same app (same style, same region boundaries,

same framing). The user picks one in intake and can switch in Settings; the engine does not care which is selected.

Male source (default): the Z-Anatomy Blender file — an open-source 3D anatomy atlas (CC BY-SA 4.0) built on the BodyParts3D dataset from Japan's Database Center for Life Science (also CC BY-SA). Every muscle is a separate object named per Terminologia Anatomica, which is exactly what this pipeline needs. Download it from the Z-Anatomy project site; record the exact revision and license text in `docs/DECISIONS.md` and `assets/models/LICENSE-model.md`.

Female source — three paths, in this order:

- **Path A (default; fully automated; ships in Phase 1 no matter what):** derive the female body from the same Z-Anatomy muscle set with a deterministic anthropometric morph (step 6 below). Same topology, same region meshes, identical style — the female model cannot drift from the male one. Parameters live in `female-morph.json`. It is a stylized proportional model, and the About screen says so; it is not marketed as sex-specific medical anatomy.
- **Path B (free upgrade; time-boxed to one session inside Phase 1):** a CC BY (attribution-only) female muscular-system model from Sketchfab — for example Ruslan Gadzhiev's "Female Body Muscular System – Anatomy Study" (~250k triangles, CC Attribution). If its muscles are separate objects, map them in `region-objects-female.json` like the male. If it is one merged mesh, run the label-transfer step (step 7) to cut it into regions. Adopt it only if it passes the visual acceptance check; otherwise stay on Path A and log why.
- **Path C (paid; humans decide):** a purchased scan-based male + female écorché pair from one vendor (e.g. the 3DScanStore Male and Female Écorché Bundle, which ships individual decimated muscle-group meshes; or a TurboSquid / CGTrader female muscular system, typically $150–600) after the humans verify the license permits distribution inside a mobile app. Buying a matched pair from one vendor keeps both bodies consistent. Same pipeline, new `region-objects-*.json`.

Steps (`build.py` , Blender headless or `bpy` wheel; deterministic; runnable in CI; run once per body):

1. Open the source; resolve objects listed in `region-objects-<sex>.json`. Entry shapes:

- `{ "region": "quads_rf", "objects": ["Rectus femoris"] }` — one or more named objects (left + right).
- `{ "region": "chest_upper", "split": { "object": "Pectoralis major", "axis": "z", "range": [0.70, 1.00] } }` — bisect a single muscle by bounding-box fraction. Required wherever the source ships one object that spans several regions; expect this for pectoralis major (upper/mid/lower), deltoid (front/side/rear — split along the anterior–posterior axis), trapezius (upper vs mid/lower), and possibly triceps if the heads are not separate objects. Check the source first: quads, hamstring heads, gastrocnemius/soleus, glute max/med, brachialis are separate muscles and should not need splitting.
- Unmapped superficial muscles, bones, and context → `body_base`.

2. Join per region; apply transforms; recalculate normals; weld/remove doubles; decimate (planar then collapse) to the per-mesh share of the 150k budget; `body_base` decimated hardest.
3. Assign one neutral gray material to every mesh; strip textures, modifiers, animations, and stray empties.
4. Normalize: origin at the model's center, feet at y = 0, stature scaled to exactly 1.0 unit so both bodies use identical camera framing and gesture math.
5. Export GLB per the asset contract; write `model.manifest.json`; render front/back/left/right PNG snapshots to `docs/models/<sex>-*.png` (committed) so the humans review every change to either body in the PR.
6. **Female morph (Path A).** Applied to the male muscle set before decimation, driven by `female-morph.json`:
   - A piecewise-linear width profile `s(y)` keyed to landmark heights as fractions of stature (shoulders ≈ 0.82, chest ≈ 0.72, waist ≈ 0.61, hips ≈ 0.52, knees ≈ 0.28, ankles ≈ 0.04): scale each vertex's x/z distance from the body's midline by `s(y)`. Starting values: shoulders ×0.88, chest ×0.92, waist ×0.95, hips ×1.08, knees ×0.98, ankles ×0.96.
   - Muscle bulk: shrink each muscle mesh radially toward its limb/torso axis — upper body ×0.86, lower body ×0.93 (women carry proportionally more lower-body muscle).
   - Proportions: stature is normalized anyway, but torso ×1.02 and legs ×0.98 along y; neck girth ×0.90; hands and feet ×0.92.
   - Add a `breast_tissue` form to `body_base` (two blended ellipsoids over the mid/lower pec, non-tracked, gray) — an écorché shows mammary tissue as non-muscle over pectoralis major.

    - These are starting values to tune against the snapshot renders, not anthropometric truth; keep them in the JSON, never in code. Male and female share `region-objects-male.json`, so region boundaries are guaranteed identical.

7. **Label transfer (Path B, only for a merged-mesh source).** Align the female mesh to the male model (normalize stature, center, face −z). Sample points on every male region surface and build a KD-tree tagged with region ids (`mathutils.kdtree`). For each female face, look up the nearest tagged sample within a distance threshold (≈ 2% of stature) and take its region; then run three passes of majority-vote smoothing over adjacent faces; unlabeled faces → `body_base`. Split by label, then continue at step 2. **Acceptance:** every region is one or two connected components (left/right), no region under 50 faces, and the four snapshot renders show clean boundaries at pec/delt/triceps and quad/hamstring seams. Fail any of these → Path A stays.
8. `tools/model-pipeline/validate.ts` (runs in CI, per body): every id in `REGION_IDS` exists as a mesh; no unmapped meshes; total triangles ≤ 150k; file ≤ 8 MB; stature normalized; bounding box sane; both bodies expose the identical region set. CI fails if either asset drifts from the contract.

License compliance (CC BY-SA 4.0 for anything derived from Z-Anatomy; CC BY for a Path B model): attribution in Settings → About ("Anatomy models derived from Z-Anatomy and BodyParts3D (DBCLS), CC BY-SA 4.0" plus the Path B author if used, with links); `LICENSE-model.md` shipped beside the assets; derived GLBs and this pipeline published in the repo under the matching licenses. Share-alike attaches to the *model files*, not to the app's code. If the humans want proprietary models later, that is Path C.

6.4 Report (weekly)

Rolling 7 days (or the program week if scheduled). Contents, all deterministic from the engine: per-group table (status, effective sets, target, sessions); **top 3 fixes ranked by impact** ("Add 4 sets of seated leg curl on Thursday — hamstrings 5/12, BF short head untrained"); junk-volume and distribution flags; balance flags; stalled lifts with a suggestion (add a rep, add load, swap variation, check RIR); adherence. Optional LLM narration ≤ 120 words, plain and direct, no hype, no emoji — the JSON is the source of truth and is what gets rendered if the narration call fails. Shareable as an image.

6.5 History

Calendar heatmap + list; workout detail; edit; duplicate as new workout; filter by exercise or group.

6.6 Videos

Allowlisted channels only (channel IDs in `src/data/video-channels.json`; initial list decided by the humans — expected: Jeff Nippard, Sean Nalewanyj, Renaissance Periodization, Stronger By Science, Jeremy Ethier). Edge function searches YouTube Data API v3 by exercise name and by muscle group, filters to the allowlist, ranks by allowlist priority then view count, caches 30 days. Hand-picked overrides in `src/data/videos.json` win over search results. Show nothing rather than an un-vetted video. Quota is limited — never call the API from the client, never search on every render.

6.7 Progress

e1RM per exercise over time; weekly effective sets per group (stacked bars); bodyweight; PR list; streaks.

6.8 Settings

Level; body model (female / male); units; target overrides per group (defaults + rationale shown); priority and optional groups; rest-timer defaults; "Muscle status (list)" — the same statuses as a grouped list (accessibility path and GL-failure fallback); About with model attribution; export CSV/JSON; backup/restore; delete all data.

---

7. Engineering conventions

- Layout: `app/` (routes) · `src/engine` (pure) · `src/db` (schema, migrations, repositories) · `src/features/<feature>` · `src/components` · `src/data` (seed JSON) · `src/services` (youtube, llm, sync) · `docs/`.
- Engine: no React imports, `now` injected, exhaustive unit tests including the golden cases in §8, constants with cited comments.
- Recompute muscle status incrementally on set save; never block the logging UI on it.
- 3D: the scene graph is built once; status updates mutate material colors (never re-mount meshes); gesture math runs on the UI thread via

Reanimated worklets; no per-frame allocations; profile on a real Android device, not only the emulator.

- All long lists virtualized. Cold start to a loggable workout < 2 s on a mid-range Android device.
- Strings in one file (English only in v1, i18n-ready).
- App copy: plain, direct, second person, no hype, no emoji.
- No analytics or third-party SDKs in v1 beyond what §3 lists.
- Accessibility: dynamic type, ≥ 44 pt touch targets, status conveyed by color + icon + text.

---

8. Phases and definitions of done

The **first build** is Phases 0–5. Phases 6–7 are the second build.

Phase 0 — Scaffold + 3D spike. Expo app with router, tabs, theme, DB + migrations, seed import script (exercises + mappings), CI green, Android dev build installs on the Pixel. The spike, in the same phase: load any test GLB with ≥ 30 named meshes (a temporary primitives model is fine here — this is the only place a placeholder is allowed), drag-rotate with inertia, pinch zoom, tap-to-select with highlight, `frameloop="demand"`, on the physical device. Hard time-box: two sessions. If pinned fiber v9 native won't run cleanly on the current Expo SDK, switch to the WebView fallback and write the ADR. DoD: spike at 60 fps on the Pixel; rendering decision logged; `MuscleMapView` props frozen.

Phase 1 — Model pipeline + Map screen. Build `tools/model-pipeline`, produce `body-male.glb` from Z-Anatomy and `body-female.glb` via Path A (then spend at most one session on Path B), pass `validate.ts` for both in CI, commit the snapshot renders, render the selected body in the Map tab with mock statuses, full §6.2 interaction spec, snap buttons, group/region toggle, female/male switch, bottom-sheet shell, attribution in About. DoD: every region tappable and colorable on both bodies; budgets met on both; switching bodies keeps the camera and selection; screen recordings of rotate → tap → sheet on the Pixel for each body.

Phase 2 — Tracker (Strong parity). DoD: a full PPL week can be logged from templates with rest timer, previous performance, supersets, warm-ups, PRs, history calendar, edit/delete, CSV/JSON export; Maestro flow "start template → log 3 exercises → finish" passes.

Phase 3 — Engine + wiring. Engine, status cache, map colored from real data, full bottom-sheet content, finish-workout recolor. DoD: engine tests pass including golden cases:

1. Beginner runs the Full Body 3× starter template for a week → every default group ≥ `partial`, all major groups `covered`.
2. Only bench press 5×5 once, RIR unlogged → chest `partial` / `low` per numbers, delt_front and triceps `low`, everything else `untrained`; `hamstrings_bf_short` and `quads_rf` untrained.
3. 16 hard sets of chest in one session → session factor applied (8 + 4×0.5 + 4×0.25 = 11 effective), `distributionAdvice` fires.
4. Last chest session 8 days ago → recency taper applied (≈ 0.67); 11 days ago → 0.
5. Provisional sets color the map but never appear in PRs or Progress.
6. Every mapping validates (weights in range, distributions sum to 1).
   Plus: the model colors correctly from real logged data, tap → detail sheet with real history, finish-workout recolor works, provisional data renders desaturated.

Phase 4 — Intake + Report. DoD: full onboarding in ≤ 3 minutes including the priority-muscle picker on the 3D model; free-text program parse round-trips to editable templates; provisional stimulus generated; weekly report renders from engine JSON with LLM narration optional and gracefully absent.

Phase 5 — Videos. DoD: edge function deployed, allowlist enforced, cache working, player embedded in the muscle detail sheet, zero client-side API calls. **End of first build:** EAS builds for both platforms, TestFlight + Play internal testing, `docs/STATUS.md` current.

Phase 6 — Accounts + sync. Supabase auth (email + Apple + Google), local-first sync of profile/templates/workouts/sets with `updatedAt` last-write-wins per row, tombstones for deletes, conflict tests. Photos are never synced in this phase.

Phase 7 — Progress photos + polish + store readiness. On-device photo timeline with side-by-side compare (no analysis); privacy policy; screenshots; App Store / Play listings; `eas submit` to TestFlight and Play internal testing.

---

9. Inputs owed by the humans — ask for these at the start of the phase that needs them

- App name (Phase 0).
- Review of the full exercise → muscle mapping table (Phase 3) and of `region-objects.json` (Phase 1).
- Video channel allowlist and any hand-picked videos (Phase 5); YouTube Data API key (server side only).
- Anthropic API key (server side only) (Phase 4).
- Supabase project (Phase 4 for edge functions; Phase 6 for auth/sync).
- Model source decisions (Phase 1 start): male defaults to the Z-Anatomy-derived model, female to Path A (morph) with Path B attempted — free, but CC BY-SA means attribution in-app and the derived model files stay share-alike. Path C is a purchased matched female + male écorché pair with an app-distribution license — proprietary and probably prettier, and the humans have to buy and license-check it. Do not block on any of this: start with Z-Anatomy and Path A, and show the humans the snapshot renders of both bodies at the end of Phase 1 so they can decide whether Path C is worth paying for.
- Apple Developer and Google Play developer accounts; EAS account (needed by the end of Phase 5 for the first build's TestFlight / Play internal testing).

On photo analysis: the original idea was to analyze uploaded progress photos during intake to seed the map. Do not build that. A vision model cannot reliably grade hypertrophy status per muscle from a phone photo, and seeding a training-coverage map from body appearance mixes two different things (how developed a muscle looks vs. whether the program is training it). The map is seeded from training history and program import; the user self-selects priority muscles; photos come back in Phase 7 as a progress timeline only. If the humans still want photo-based priority suggestions later, it must be a separate, clearly labeled "suggestion" that never sets a color.